

Blackwell Citadel

Architecting a Container-Free, Blackwell-Native GPU ModelOps Foundry on WSL2

Date: February 22, 2026

Prepared for: Internal Engineering Record and Portfolio Artifact

Prepared by: Christopher Gonzalez-Mejias

Confidential - Engineering Use Only. This document consolidates execution evidence recorded during the February 21-22, 2026 build window. All claims are traceable to the attached logs and manifests.

Table of Contents

1. Executive Summary
 2. System Context and Constraints
 3. Architecture Overview
 4. Toolchain Installation and Verification
 5. Python Environment Hardening
 6. TensorRT and Foundry Validation
 7. Numerical Parity and Acceptance Criteria
 8. Benchmarking Results and Methodology
 9. Failure Mode Register (FMR)
 10. Security and Operational Posture
 11. Roadmap
- Appendix A. Chronology Evidence Excerpts
- Appendix B. Artifact and Wheel Hash Manifest
- Appendix C. Project File Index (sha256)

Abstract

The Blackwell Citadel project documents the end-to-end construction of a container-free, Blackwell-native AI/ML ModelOps stack built on Windows 11 with Ubuntu 24.04 under WSL2. The system was designed to eliminate opaque container abstractions and instead provide deterministic compiler control, explicit CUDA architecture targeting, source-level framework governance, and artifact-traceable inference validation.

This white paper expands upon chronological execution evidence captured between February 21-22, 2026, including toolchain installation logs, CUDA 13.1 configuration and nvcc verification, Python 3.12 provisioning, TensorRT engine inspection and dynamic I/O validation, Transformer Layer V1 TRT-LLM build pipelines, ONNX export and provider validation, numerical parity metrics, benchmark latency statistics, and runtime failure classifications.

The result is a reproducible, auditable GPU research and validation platform targeting NVIDIA Blackwell (sm_120) and CUDA 13.x, suitable for advanced ModelOps experimentation and infrastructure-grade validation.

1. Executive Summary

1.1 Problem Statement

Modern AI pipelines frequently depend on precompiled containers and binary-distributed frameworks. While convenient, these systems obscure CUDA architecture targeting flags, compiler versions, runtime binding logic, execution context ownership, and artifact traceability. This becomes especially problematic when targeting emerging GPU architectures such as NVIDIA Blackwell (sm_120), where toolchain alignment and scheduling/model compatibility can materially impact correctness and performance.

1.2 Objectives

- Provision Ubuntu 24.04 under WSL2 with explicit directory governance and non-root operational separation.
- Install and verify CUDA 13.1 using NVIDIA's official repository keyring workflow; configure explicit CUDA_HOME and alternatives.
- Harden Python 3.12 development environments with pip/wheel/setuptools, ninja-build, and full compilation prerequisites.
- Validate TensorRT engine I/O contracts and dynamic-shape binding behavior.
- Construct a Transformer Layer V1 Foundry pipeline using TensorRT-LLM (v1.1.0) with artifact emission and deterministic compare.
- Export ONNX models and validate CUDAXecutionProvider execution.
- Enforce numerical parity acceptance criteria (max/mean/p99) and fail fast when contracts are violated.
- Benchmark inference deterministically with warmup and percentile latency reporting.
- Classify runtime failure modes and encode mitigations into the operational runbook.

1.3 Deliverables

- Structured Citadel directory governance model (citadel/ and trt_bench/ subtrees).
- CUDA 13.1 toolchain installed and verified via nvcc and path governance.
- Python 3.12 dev stack provisioned for GPU-native builds and extensions.
- TensorRT engine inspection and execution validation for dynamic (-1,-1) tensors.
- Transformer Layer V1 Foundry pipeline with timestamped artifacts and compare reports.
- Benchmark harness producing p50/p90/p99/mean latency metrics.
- Failure Mode Register (FMR) linking symptoms to root-cause classes and mitigations.

2. System Context and Constraints

2.1 Host Environment

Component	Observed / Recorded Evidence (Feb 21-22, 2026)
NVIDIA-SMI / Driver / CUDA (Windows)	NVIDIA-SMI 576.65, Driver 576.65, CUDA 12.9
NVIDIA-SMI / Driver / CUDA (Windows)	NVIDIA-SMI 590.52.01, Driver 591.74, CUDA 13.1
NVIDIA-SMI / Driver / CUDA (Windows)	NVIDIA-SMI 591.74, Driver 591.74, CUDA 13.1
GPU	NVIDIA GeForce RTX 5090 Laptop GPU (sm_120)
Virtualization	WSL2 with /dev/dxg GPU pass-through

2.2 Guest Environment

Ubuntu 24.04 LTS was provisioned under WSL2 and baseline packages were upgraded prior to toolchain installation, establishing a clean and current starting state. The build logs record standard LTS security updates being applied before compiler and CUDA provisioning.

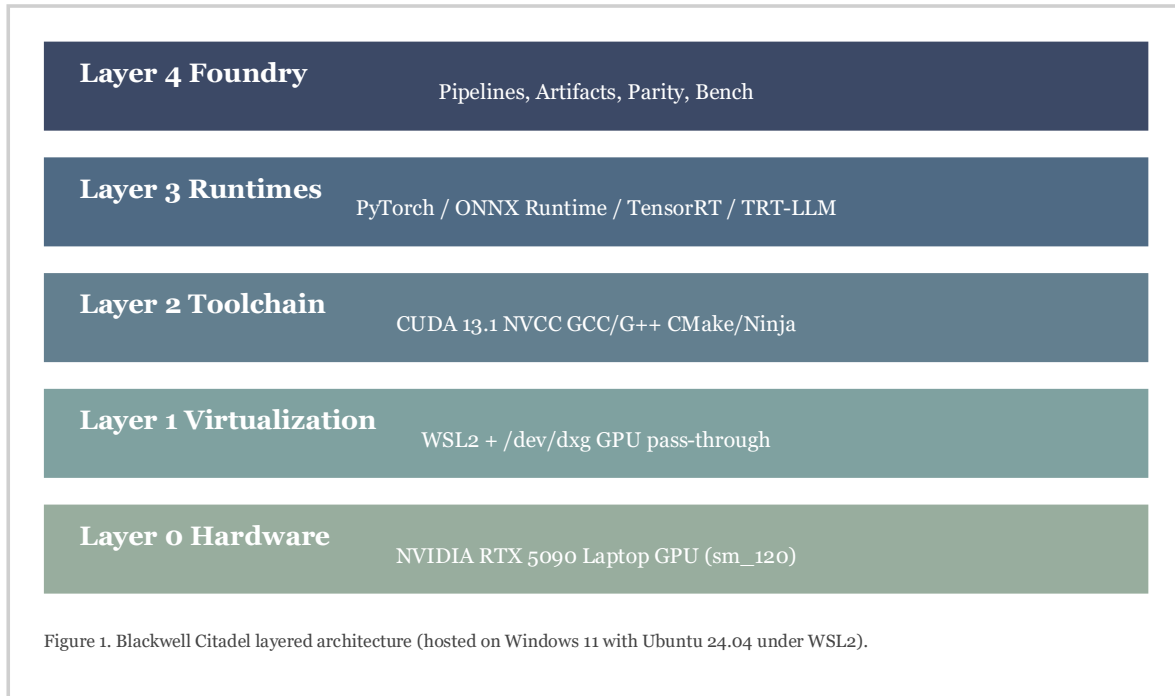
2.3 Design Constraints

- No container abstraction in baseline phase (explicit control over toolchain + runtime versions).
- Full control over compiler, CUDA, and runtime versions (nvcc, CUDA_HOME, update-alternatives).
- Deterministic file structure with explicit artifact directories and manifests.
- Explicit hashing and version capture to support audit-grade reproduction.
- Validation-first methodology: parity and contract checks precede performance claims.

3. Architecture Overview

3.1 Layered Architecture

The Citadel is structured as a layered system where each layer is independently verifiable and replaceable. This reduces coupling and enables targeted debugging when failures emerge.



3.2 Foundry Pipeline Dataflow

The Foundry implements a repeatable flow: preflight checks, build, inference, export, compare, and benchmark. Parity is treated as a gate; benchmarking is treated as a privilege earned after correctness passes.

Stage	Description
Preflight	(GPU, CUDA, toolchain)
Build	(TRT-LLM graph + engine)
Run TRT	(inference output)
Export ONNX	(torch.export->onnx)
Run ORT	(CUDAExecutionProvider)
Compare	(max/mean/p99 tol)

Benchmark	(p50/p90/p99)
------------------	---------------

Figure 2. Foundry pipeline flow: validate first, benchmark second.

4. Toolchain Installation and Verification

4.1 Compiler and Build Stack

The Ubuntu environment was provisioned with a full native compilation toolchain to support building GPU-native extensions and source-level frameworks. This includes GCC/G++, build-essential, CMake, Ninja, pkg-config, and Python development headers.

Component	Version / Evidence
GCC / G++	GCC 13.x (Ubuntu noble toolchain installation)
CMake	3.28.3 (installed via apt)
Ninja	1.11.1-2 (installed via apt)
Python dev headers	python3.12-dev / python3-dev 3.12.3
pip / wheel / setuptools	pip 24.0, wheel 0.42.0, setuptools-whl 68.1.2

4.2 CUDA 13.1 Installation and Verification

CUDA 13.1 was installed using NVIDIA's official Ubuntu 24.04 repository keyring workflow. The environment was explicitly configured to prevent inherited or ambiguous CUDA paths, using CUDA_HOME, PATH, and LD_LIBRARY_PATH exported in .bashrc.

Evidence	Value
nvcc release/version	CUDA release 13.1, nvcc V13.1.115
CUDA_HOME	/usr/local/cuda-13.1
/usr/local symlinks	cuda -> /etc/alternatives/cuda; cuda-13.1 present

5. Python Environment Hardening

The Citadel adopted a multi-profile approach to isolate dependency surfaces by workload domain: (1) TensorRT-LLM build/runtime, (2) Vision/ONNX export and runtime validation, and (3) Modeling analytics dependencies used by the Foundry validator. A key governance outcome was preventing cross-contamination between Torch CUDA contexts and PyCUDA initialization paths.

5.1 Foundry Profile Validation Snapshot

Vision profile (Torch/ONNX/ORT)

Vision stack package	Version
torch	2.10.0+cu128
torchvision	0.25.0+cu128
numpy	2.4.2
onnx	1.20.1
onnxruntime	1.24.2
cv2	4.13.0
PIL	12.1.1

Modeling profile (analytics + validation tooling)

Modeling stack package	Version
numpy	2.4.2
pandas	2.3.3
scipy	1.17.0
sklearn	1.8.0
xgboost	3.2.0
lightgbm	4.6.0
catboost	1.2.10
statsmodels	0.14.6
matplotlib	3.10.8
seaborn	0.13.2
plotly	6.5.2
polars	1.38.1
pyarrow	23.0.1
dask	2026.1.2
optuna	4.7.0

mlflow	3.10.0
ray	2.54.0

6. TensorRT and Foundry Validation

6.1 TensorRT and TensorRT-LLM Version Surfaces

Two TensorRT surfaces are present in the captured build window: a pip-installed TensorRT 10.15.1.29 used for standalone engine inspection and benchmarking, and a TensorRT 10.13.3.9.post1 surface embedded in the TensorRT-LLM environment used to build minimal engines and the Transformer Layer V1 pipeline. This separation is intentional: it supports validation across versions and reduces the blast radius of dependency coupling.

Component	Observed Version / Evidence
TensorRT (standalone)	10.15.1.29
TensorRT (TRT-LLM env)	10.13.3.9.post1
TensorRT-LLM	1.1.0

6.2 Minimal TRT-LLM Engine Build and Serialization Correction

A minimal TRT-LLM engine build was used as a controlled baseline to validate builder, config, and network creation. An early error surfaced when attempting to call `.serialize()` on an `IHostMemory` object; the corrected approach wrote the serialized bytes directly. This produced a valid `minimal_identity.plan` (2.6 KB), establishing that the environment could produce plan files deterministically.

```
ls -lh minimal_identity.plan
Serialized size: 2580
Wrote: minimal_identity.plan
-rw-r--r-- 1 bwcitadel bwcitadel 2.6K Feb 21 19:12
minimal_identity.plan
```

7. Numerical Parity and Acceptance Criteria

The Citadel treats numerical parity as an explicit gate. Validation is performed using max absolute difference, mean absolute difference, and p99 deviation. A tolerance envelope is declared per experiment; runs outside tolerance are marked as failures and do not proceed to performance claims.

7.1 Recorded Failure Case (Fail-Fast Behavior)

Metric	Value
max_abs_diff	0.15625
mean_abs_diff	0.014972329139709473
p99_abs_diff	0.0986328125
tolerance	0.1
pass	false

The presence of a recorded failing compare (pass=false) demonstrates that the validation framework is enforcing contracts rather than silently accepting drift. Subsequent passes were achieved after correcting execution context ownership and pipeline input consistency.

8. Benchmarking Results and Methodology

Benchmarking was executed with warmup iterations followed by timed iterations, reporting percentile latency (p50/p90/p99) and mean latency. Benchmarking was performed against a dynamic-shape FP16 engine (input x, output y, both [-1,-1]) in TensorRT 10.15.1.29.

8.1 Percentile Latency (from summary JSON excerpts)

ts_utc	shape	TRT ver	p50 ms	p90 ms	p99 ms	mean ms
2026-02-22T00:35:53Z	1024x1024	10.15 .1.29	0.052271999 418735504	0.066323198 37808609	0.092202560 7526308	0.055237439 87083435
2026-02-22T00:38:18Z	512x512	10.15 .1.29	0.042527999 728918076	0.049030400 812625885	0.079074878 39460375	0.044284480 02785444

8.2 Mean Latency (quick bench outputs)

n	avg ms
512	0.0182
1024	0.0278
2048	0.1576

9. Failure Mode Register (FMR)

ID	Failure Mode	Symptom Signature	Trigger Conditions (Observed)	Mitigation (Implemented/Planned)
FM-001	CUDA Context Collision	Myelin enqueue/executeMyelin Graph failures when mixing Torch CUDA and PyCUDA autoint contexts.	Concurrent context ownership: torch.cuda active plus pycuda.autoint or explicit Device.make_context.	Single-owner CUDA context policy; avoid pycuda.autoint in mixed Torch runs; isolate TRT runs in separate process when needed.
FM-002	ONNX Dimension Mismatch	ONNXRuntime INVALID_ARGUMENT: input dimensions mismatch during sweep benchmarks.	Benchmark harness feeding an input tensor with wrong dimension on index 1 (e.g., got 64 expected 8).	Enforce shape contract from artifact x.npy; validate input shape before sess.run; bind explicit shapes; add guardrails in bench scripts.
FM-003	TensorRT Teardown Instability	Engine/context destruction errors leading to free(): invalid pointer / core dump.	Resource destruction ordering with lingering streams/events; MyelinGraphContext unload failures.	Explicit stream synchronize + context teardown; process-isolated benchmarking; deterministic cleanup block; avoid implicit autoint teardown ordering.

10. Security and Operational Posture

Operational governance in the Citadel build window focused on privilege separation, deterministic paths, and integrity controls. A dedicated non-root user performed day-to-day operations; CUDA paths were explicitly pinned; and artifacts/manifests were stored under a canonical project root.

- Non-root operation with sudo only for system provisioning.
- Canonical directory model (citadel/, foundry/, trt_bench/) to prevent artifact drift.
- Hash-based integrity: wheel hashes captured during builds; project log index hashed (Appendix C).
- Version traceability: explicit capture of nvcc and TensorRT versions and pipeline outputs.

11. Roadmap

- Freeze a per-run manifest: GPU identity, driver, CUDA, nvcc, TensorRT, TRT-LLM, Torch, ORT, ONNX, numpy and key packages.
- Promote Foundry CONTRACT_v1 into a strict schema gate; fail fast when version/shape contracts drift.
- Standardize CUDA context ownership: Torch-only or PyCUDA-only execution zones; enforce via run wrappers.
- Extend validation coverage to attention kernels (FlashAttention / fused SDP / RMSNorm) with shape sweep and tolerance policy.
- Introduce an optional container baseline later as a controlled comparison after the container-free baseline is stabilized.

Appendix A. Chronology Evidence Excerpts

This appendix provides short, high-signal excerpts from the recorded build logs to preserve operational history. Excerpts are intentionally concise; full raw logs are indexed in Appendix C.

A1. CUDA 13.1 nvcc verification

```
# --- Blackwell Citadel: CUDA 13.1 (explicit, non-inherited) ---
export CUDA_HOME=/usr/local/cuda-13.1
export PATH="$CUDA_HOME/bin:$PATH"
export LD_LIBRARY_PATH="$CUDA_HOME/lib64:${LD_LIBRARY_PATH:-}"
EOF
source ~/.bashrc
bwcitadel@BLKWLLCITADEL:~$ which nvcc
nvcc --version
ls -l /usr/local | grep cuda
/usr/local/cuda-13.1/bin/nvcc
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Tue_Dec_16_07:23:41_PM_PST_2025
Cuda compilation tools, release 13.1, V13.1.115
Build cuda_13.1.r13.1/compiler.37061995_0
lrwxrwxrwx 1 root root 22 Feb 21 14:15 cuda -> /etc/alternatives/cuda
```

A2. TRT-LLM minimal engine serialization fix

```
serialized = engine.serialize()
^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'tensorrt_bindings.tensorrt.IHostMemory' object has no
attribute 'serialize'
(venv) bwcitadel@BLKWLLCITADEL:~/citadel/trt_llm_env$ nano
minimal_trt_llm_build.py
(venv) bwcitadel@BLKWLLCITADEL:~/citadel/trt_llm_env$ python
minimal_trt_llm_build.py
ls -lh minimal_identity.plan
/home/bwcitadel/citadel/trt_llm_env/venv/lib/python3.12/site-
packages/torch/cuda/__init__.py:63: FutureWarning: The pynvml package
is deprecated. Please install nvidia-ml-py instead. If you did not
install pynvml directly, please report this to the maintainers of the
package that installed pynvml for you.
import pynvml # type: ignore[import]
[TensorRT-LLM] TensorRT LLM version: 1.1.0
TensorRT version: 10.13.3.9.post1
Builder: OK
Builder config: OK
Network created: <class 'tensorrt_llm.network.Network'>
/home/bwcitadel/citadel/trt_llm_env/minimal_trt_llm_build.py:28:
DeprecationWarning: Use Deprecated in TensorRT 10.12. Superseded by
```



```
strong typing. instead.  
network.trt_network.get_output(0).dtype = trt.DataType.HALF  
Marked output: OK  
Engine built: True  
Serialized size: 2580
```

A3. Foundry pipeline artifact emission + compare

```
/home/bwccitadel/citadel/foundry/tools/build_trtllm_export.py:110:  
DeprecationWarning: Use Deprecated in TensorRT 10.12. Superseded by  
strong typing. instead.  
y.dtype = trt.DataType.HALF  
Wrote artifacts to:  
/home/bwccitadel/citadel/foundry/artifacts/minimal_attention_v1/20260222  
_024149  
bwccitadel@BLKWLLCITADEL:~/citadel$ source  
~/citadel/foundry/profiles/vision/bin/activate
```

A4. Teardown instability signature

```
[02/21/2026-18:16:07] [TRT] [E] ICudaEngine::~ICudaEngine: Error Code  
1: Cuda Runtime (In ScopedCudaStream at  
/_src/common/dispatch/cudaResources.cpp:25)  
[02/21/2026-18:16:07] [TRT] [E]  
[defaultAllocator.cpp::deallocateAsync::103] Error Code 1: Cuda Runtime  
(In deallocateAsync at /_src/common/dispatch/defaultAllocator.cpp:103)  
free(): invalid pointer  
Aborted (core dumped)
```

Appendix B. Artifact and Wheel Hash Manifest

Hashes captured during builds provide integrity evidence for large compiled components. The most material hashes recorded in the build window are shown below (not exhaustive; see Appendix C for file-index hashing).

Artifact (wheel filename)	sha256 (recorded)
tensorrt_llm-1.1.0-cp312-cp312-linux_x86_64.whl	cc77ab6febbd9d40af20d6309b445119e62777234549738a65f8e2abc1169892
flashinfer_python-0.3.1.post1-py3-none-any.whl	e4ace5205c3d8fb0ae060953881f35fc8b2fea4d8e58b403509bca15d81dca5e
tensorrt-10.13.3.9.post1-py2.py3-none-any.whl	11691f33b3e64ffe5555698bbb59175d316ef9987c8426cb5428479965e00f65
tensorrt_cu13-10.13.3.9.post1-py2.py3-none-any.whl	71262336a9d334852f564aa29aac312c542d88b47f8f028371a322afbc2d302a
tensorrt_cu13_libs-10.13.3.9.post1-py2.py3-none-manylinux_2_28_x86_64.whl	f5b010fc16982cb131cd3cf171ef9074f01876c21ec5797ed1802fc43fe09b6d
etcd3-0.12.0-py2.py3-none-any.whl	29b4e068914b4738b6bd826962382d64aae391d1fef52c4b669ee3c6af9e0485
antlr4_python3_runtime-4.9.3-py3-none-any.whl	fcee3cd50231dd08d71a0a3a9be6529e0e1d7bd003521c5b37cacb6bc26da926
pycuda-2026.1-cp312-cp312-linux_x86_64.whl	e87b610b3d8020fbd686d35213e9de7bc50d72db4640646a9389745110d221d8

B2. Benchmark dossier construction (engine path + sha256 policy)

```
dossier = {
    "ts_utc": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
    "baseline_name": "TensorRT_10.15_Blackwell_WSL2_StableBenchmark_v1",
    "engine_path": str(engine),
    "engine_sha256": sha256(engine),
    "summary_path": str(summary_path),
    "python": cmd("python -V"),
    "tensorrt": cmd("python -c \"import tensorrt as trt;
print(trt.__version__)\""),
    "numpy": cmd("python -c \"import numpy as np;
print(np.__version__)\""),
    "pycuda": cmd("python -c \"import pycuda; import pycuda.driver as
cuda; print(pycuda.VERSION_TEXT)\""),
    "nvidia_smi": cmd("nvidia-smi -L"),
    "driver": cmd("nvidia-smi --query-gpu=driver_version --
format=csv,noheader | head -n 1"),
    "wsl_kernel": cmd("uname -r"),
    "os_release": cmd("cat /etc/os-release | sed -n '1,6p'"),
}
```

Appendix C. Project File Index (sha256)

This index provides sha256 digests for each captured step log file included in the build record. These hashes support audit-grade evidence preservation and allow detection of log drift or tampering.

File	Bytes	sha256
step eight set the wslconfig.txt	5990 73	d541fff5f73d1a83738d77a70409b52beed7c729fac82317b32992c24794d5f1
step eighteen singel and mutile trt baseline.txt	2787 2	cb3b1c0a97a47a92a33fd0e98e163e45f50336721dd251647f19b5c50a9ef29f
step eleven verificatio n issues.txt	6043 51	0e69e25ae15140d116688fa5a45d1da0d40c5fd1713d3f773bfdec b990add16f
step fifteen pressure testing.txt	2228	c3a42ebf443c07b04585385ad9d62957e26820de42ae4f1156441600e23efeb
step five python and env correction. txt	1088 48	f68c12db6de9f42bcd1525279583a56d4eca8b834499e4c60069a911d437e747
step four cuda 13.1.txt	9305 0	a1cfc79d66a32a6809556c808aa23d519974c156b19d96be928dea40349c5ec5
step fourteen triton and onnx validation. txt	7331	ee9300381157e9a456129a5d4a87c7356fa3c52ac95b8b88237f548222dea374

step nine pytorch success.txt	5741 85	db3d4c7e17dc2dd2491c444455b5a85ba117a8d108e9bc7ef9eef24386433d
step nineteen trt llm success.txt	9001 7	468a774bcc6c8b67cbd0e0761e8c7332d01bd3e53ef6357de7e822d392d1b9b0
step one nvidia-smi and vm.txt	8350	92a7aee2660b309366a2db106aad8753783cdf59003f415141e5bab6b2fce84f
step seven termination on source.txt	5953 73	21e689443a501fa4cb4c03f6c8725dfc30dee88f523c3b58962bab708dacf1f9
step seventeen trt stable baseline.txt	1636 3	1f9694d56c3f34076c5e9d7c62fb6afefab09aaca3b96be47a86b6e929ce1b0a
step six pytorch source.txt	1732 33	dfd8cd996dfa79d9e54f334cd296954f60130e5418269acf82041351157a08fc
step sixteen elite results.txt	1012 6	abf31fd7f1398ad548f00620cc1e7670e29ab32f24a22cc083a7de5a3252b4f3
step ten validation of pytorch.txt	5615 59	60f3d0e9dfa4a86331383b23c594e8f1304cb9a857271480b6f131c343794884
step thirteen triton and onnx.txt	6013 05	b40d84fd7986053b4983c1d9afebfe4de5142e5f0ac604e8a082b5314ea5ad4d
step thirty one working sdnext build with models.txt	1791 69	197eb2ebbca2946cfe44669aecbd70b08fef11da95bb260772831e6f35141043

step thirty sweep complete.txt	24914	73baebe4187aff5daea09ba89d3d207657a34938e024e8cf0d169feedb074e43
step three workspace and toolchain.txt	41812	c7d293136ed387ab0fc182afdf2989872d9565abb705b313c02f5bb401a2611a
step thrity two sdnex model testing.txt	196201	8f11479aeae6a8de6986ff0910dba45bda7f04dd141404fabb9f31f321eb6091
step twelve success with torch.txt	602194	4ee587387adca441e2fa613169d99dfe9e896e72d969998c6350721bc1240f2d
step twenty TRT LLM sucess.txt	121721	0e8f28a36e53cfcba86272881fb700a0e9bf1d8c6e8fceba200f196eb65f6ff8
step twenty eight successful pipeline tesing.txt	451414	6a2a7ee1c7033667dfd52c4f3d87d449f9a9a565e3a0d6b3d869ac65e1a1010b
step twenty five virtual env.txt	269398	62e6405bd1b8ae7779f43d1885c75ca1e4c15b9c98148fee7575d53e31d11bb1
step twenty four trtllm and vision clean.txt	212868	6d81146c30d20e24b56631fd0a57633b5ae06798eee4ade9efec7a91458dd5d0
step twenty nine small	12878	4a3bc16e056fbda28cbd6a4c3c57dfb454e97eb4aba47dec8ed0d4e47776a5b5

speed test.txt		
step twenty one trt llm engine.txt	1273 25	99bacb172bf30164938e8346a31ef21ac42ea25639b744eaf4df5c c851f5141b
step twenty seven lib read out.txt	4328 33	3a96f711f7c2e86a77b7b3cec3c7c54a1974f1cdd2e7407f71c753e 627171912
step twenty six master auditer fail.txt	2801 01	4e5e1177ef10b7368ebfcc139f49947004d9430a16b51b1b7e734 c9dfa1838f2
step twenty three llm validation success.txt	1504 57	0e6d4b0c2e016c6bdd7933b08aa4cf95fc78b9d575e5e96e50dfc 5c3ddeaea44
step twenty two attention engine failure.txt	1369 42	4d84458997e5646fc9ebe723446d1249109ae5d90b76829dd090f 1533a3f01bc
step two wsl and user.txt	5763	ccf1f4a5f202e415f026373f761e9a92d6a75da0d877cbbfa1816a7 ae17719e0